

REPORTE DE PRACTICA

Wifi 1 - ESP32

Tecnologías Inalámbricas

Fecha: 18/05/2026

Integrantes:

Eduardo Cadengo Lopez
Itzel Citlalli Martell De La Cruz
Damian Alexander Diaz Pina

Índice

1. Investigación: ESP32	1
2. Investigación: Portal Cautivo	2
3. Datos Generales	2
4. Descripción del Proyecto	2
5. Funcionamiento del Sistema	3
6. Configuración de la Red WiFi	3
7. Características del Portal	4
8. Código en Arduino IDE (ESP32)	4
9. Conclusiones	11
10. Enlaces del Proyecto	12

1. Investigación: ESP32

El ESP32 es un microcontrolador desarrollado por Espressif Systems que tiene integrada conectividad WiFi y Bluetooth en un solo chip. Es muy usado en proyectos de Internet de las Cosas (IoT) porque su costo es bajo, consume poca energía y es bastante versátil para distintas aplicaciones.

Tiene muchos pines de entrada y salida (GPIO) que te permiten conectarle sensores, pantallas, relevadores y otros módulos sin complicaciones. Además, tiene suficiente capacidad de procesamiento para correr un servidor web, manejar conexiones de red y ejecutar lógica de aplicación al mismo tiempo, todo sin necesitar hardware extra.

En esta práctica se aprovecho principalmente su modo de punto de acceso (Access Point o AP), que le permite crear su propia red WiFi a la que otros dispositivos se pueden conectar directamente. No necesita un router ni internet para que esto funcione, el ESP32 hace todo por su cuenta.

2. Investigación: Portal Cautivo

Un portal cautivo es una página web que aparece automáticamente cuando un usuario se conecta a una red WiFi, antes de que pueda navegar a cualquier otro sitio. Lo has visto en hoteles, aeropuertos, cafeterías y cualquier lugar que ofrezca WiFi gratis o controlado.

El truco está en el servidor DNS. Cuando tu dispositivo se conecta a la red e intenta cargar cualquier página, el servidor DNS intercepta esa petición y te redirige a la IP del portal en lugar de dejarte ir a donde ibas. Esto hace que el portal aparezca de forma automática sin que el usuario tenga que escribir ninguna dirección.

Una vez dentro del portal, se le puede pedir al usuario que acepte términos de uso, que ingrese sus datos, que vea un anuncio o cualquier cosa que el administrador de la red decida poner como condición para dar acceso. En esta práctica se implementó este mecanismo completamente dentro de un ESP32, sin depender de ningún servidor externo.

3. Datos Generales

Concepto	Descripción
Practica	Portal cautivo con ESP32 en modo AP
Plataforma	ESP32
Entorno de desarrollo	Arduino IDE
Conexiones externas	Ninguna
Dispositivos usados	ESP32 y computadora (o celular para pruebas)
Objetivo	Crear un portal cautivo que aparezca automáticamente al conectarse a la red WiFi generada por el ESP32

4. Descripción del Proyecto

En esta práctica se implementó un portal cautivo usando solo una ESP32 configurada en modo punto de acceso (Access Point). La idea era que cualquier dispositivo, ya sea un celular o una computadora, que se conectara a la red WiFi creada por el ESP32 fuera redirigido de forma automática a una página web que corre localmente en el mismo microcontrolador.

Para darle una temática al portal, se eligió el concepto de un hotel de lujo. Al conectarse a la red, el usuario ve una pantalla elegante que le pide dos datos:

- Nombre del huésped
- Numero de habitación

Una vez que el usuario llena el formulario y presiona el botón de conectar, el sistema genera y muestra una pantalla de bienvenida personalizada con el nombre y el número de habitación que ingreso.

Diagrama de interacción del sistema

El flujo de comunicación entre los componentes del sistema es el siguiente:

- El dispositivo del usuario se conecta a la red WiFi "Hotel_WiFi_SlxSEven".
- El dispositivo hace una consulta DNS para resolver cualquier dominio de internet.
- El servidor DNS del ESP32 intercepta esa consulta y responde con su propia IP (192.168.4.1).
- El navegador del dispositivo carga automáticamente la pagina en esa IP y muestra el portal.
- El usuario llena el formulario y lo envía. El ESP32 procesa los datos y responde con la página de bienvenida.

5. Funcionamiento del Sistema

El sistema completo funciona de manera secuencial y automática. Aquí se explica paso a paso lo que ocurre desde que alguien se conecta hasta que ve la página de bienvenida:

- El ESP32 arranca y crea una red WiFi abierta con el nombre "Hotel_WiFi_SlxSEven" y la IP 192.168.4.1.
- El usuario busca redes disponibles en su dispositivo y se conecta a esa red.
- Al conectarse, el sistema operativo del dispositivo hace una prueba de conectividad a internet. Esta petición pasa por el servidor DNS del ESP32.
- El servidor DNS responde a esa petición (y a cualquier otra) con la IP del ESP32, sin importar que dominio se haya pedido.
- El navegador del dispositivo recibe esa respuesta y carga la página del portal automáticamente.
- El usuario llena el formulario con su nombre y número de habitación y presiona "Conectar".
- El ESP32 recibe los datos, genera la página de bienvenida con el nombre personalizado y la manda al navegador.
- El ESP32 también imprime en el monitor serial los datos de quien se conectó, lo que es útil para verificar el funcionamiento durante pruebas.

6. Configuración de la Red WiFi

La red WiFi que crea el ESP32 tiene las siguientes características:

- SSID (nombre de la red): Hotel_WiFi_SlxSEven
- Tipo de red: Abierta, sin contraseña
- Dirección IP del ESP32: 192.168.4.1
- Mascara de subred: 255.255.255.0

- Puerto del servidor DNS: 53 (estándar)
- Puerto del servidor web: 80 (HTTP estándar)

Al ser una red abierta, cualquier dispositivo puede conectarse sin ingresar una contraseña. Esto es intencional para facilitar que la gente se conecte y vea el portal. Si se quisiera agregar seguridad, bastaría con poner una contraseña en la variable `PASS_RED` del Código.

7. Características del Portal

El portal cautivo implementado cumple con los siguientes requisitos de la practica:

- Temática libre aplicada: el portal simula el sistema de acceso a internet de un hotel de lujo, con diseño oscuro y elegante.
- Solicita al menos dos datos al usuario: nombre del huésped y numero de habitación.
- Muestra información relevante: velocidad de internet disponible y numero de soporte interno.
- Genera un mensaje de bienvenida personalizado usando los datos del formulario.
- El portal se abre automáticamente en el dispositivo del usuario al conectarse a la red, sin que tenga que escribir ninguna dirección.
- El diseño es responsivo y funciona bien tanto en celulares como en computadoras.

8. Código en Arduino IDE (ESP32)

Explicación general del código

El código hace tres cosas principales para lograr que el portal funcione:

- Crea la red WiFi en modo Access Point para que otros dispositivos puedan conectarse al ESP32 directamente.
- Corre un servidor DNS que redirige cualquier consulta de dominio hacia la IP del ESP32. Este es el mecanismo que activa el portal automáticamente en el dispositivo del usuario.
- Sirve las páginas web con un servidor HTTP que responde con el HTML del portal o de la bienvenida según la ruta que se pida.

Las dos páginas HTML (el portal y la bienvenida) se guardan en la memoria Flash usando la directiva `PROGMEM`. Esto es importante porque la RAM del ESP32 es limitada y no hay suficiente espacio para tener bloques de texto tan grandes ahí. La Flash tiene mucha más capacidad y es donde normalmente vive el programa.

Código completo

```
#include <WiFi.h>
#include <DNSServer.h>
#include <WebServer.h>

// Variables de configuracion del sistema
// Aqui defines el nombre de la red, la contraseña y la IP que va a tener el
ESP32
// Si quieres cambiar algo de la red, este es el unico lugar donde tienes
que hacerlo
const char* SSID_RED    = "Hotel_WiFi_SiXSEven"; // Nombre visible de la red
WiFi
const char* PASS_RED    = "";                    // Vacio significa red
abierta, sin contraseña
const IPAddress IP_AP(192, 168, 4, 1);           // Direccion IP local del
ESP32 en la red
const byte      DNS_PORT = 53;                   // Puerto 53 es el
estandar del protocolo DNS

// Objetos globales que manejan el DNS y el servidor web
DNSServer  dnsServer;
WebServer  server(80); // El servidor web escucha en el puerto 80, que es
el de HTTP

// Pagina HTML del portal cautivo almacenada en la Flash del ESP32
// Se usa PROGMEM para no gastar RAM con este bloque de texto tan largo
// La RAM del ESP32 es limitada, por eso es mejor dejar los datos grandes en
la Flash
const char PORTAL_HTML[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0,
maximum-scale=1.0">
  <title>Hotel WiFi</title>
  <style>
    *{margin:0;padding:0;box-sizing:border-box}
    body{
      background:#09090f;
      min-height:100vh;
      display:flex;
      flex-direction:column;
```

```

        align-items:center;
        justify-content:center;
        font-family:-apple-system,'Helvetica Neue',sans-serif;
        color:#e8e0d0;
        padding:24px;
    }
    .card{
        background:#12121a;
        border:1px solid #2a2620;
        border-radius:20px;
        padding:36px 28px;
        width:100%;
        max-width:360px;
        text-align:center;
    }
    .icono{
        width:58px;height:58px;
        margin:0 auto 20px;
        background:#c9a96e15;
        border:1px solid #c9a96e44;
        border-radius:50%;
        display:flex;align-items:center;justify-content:center;
    }
    .icono svg{width:28px;height:28px;fill:none;stroke:#c9a96e;stroke-
width:2;stroke-linecap:round;}
    h1{
        font-family:Georgia,'Times New Roman',serif;
        font-size:22px;font-weight:normal;
        letter-spacing:.12em;color:#c9a96e;
        text-transform:uppercase;margin-bottom:4px;
    }
    .sub{
        font-size:10px;letter-spacing:.25em;
        color:#5a5040;text-transform:uppercase;margin-bottom:28px;
    }
    hr{border:none;height:1px;background:#1e1c18;margin-bottom:24px;}
    label{
        display:block;font-size:10px;letter-spacing:.18em;
        text-transform:uppercase;color:#6a5f4a;
        margin-bottom:7px;text-align:left;
    }
    input[type=text]{
        width:100%;background:#0d0d14;
        border:1px solid #2a2620;border-radius:10px;

```

```

        color:#e8e0d0;font-size:15px;
        padding:13px 15px;margin-bottom:14px;
        outline:none;-webkit-appearance:none;
    }
    input[type=text]:focus{border-color:#c9a96e55;}
    input[type=text]::placeholder{color:#2e2820;}
    .check-row{
        display:flex;align-items:flex-start;
        gap:10px;margin-bottom:22px;text-align:left;
    }
    .check-row input{width:16px;height:16px;margin-top:3px;flex-
shrink:0;accent-color:#c9a96e;}
    .check-row span{font-size:12px;color:#5a5040;line-height:1.5;}
    .btn{
        width:100%;background:#c9a96e;color:#09090f;
        border:none;border-radius:10px;
        font-size:13px;font-weight:700;
        letter-spacing:.18em;text-transform:uppercase;
        padding:15px;cursor:pointer;
    }
    .btn:active{opacity:.85;}
    .pie{margin-top:18px;font-size:11px;color:#2e2820;letter-spacing:.04em;}
</style>
</head>
<body>
    <div class="card">
        <div class="icono">
            <svg viewBox="0 0 24 24">
                <path d="M1.5 8.5a14.5 14.5 0 0 1 21 0"/>
                <path d="M5 12a11 11 0 0 1 14 0"/>
                <path d="M8.5 15.5a6.5 6.5 0 0 1 7 0"/>
                <circle cx="12" cy="19" r="1" fill="#c9a96e"/>
            </svg>
        </div>
        <h1>Grand Hotel</h1>
        <p class="sub">Acceso a Internet &mdash; Wi-Fi</p>
        <hr>
        <form method="GET" action="/conectar">
            <label>Nombre del huesped</label>
            <input type="text" name="nombre" placeholder="Tu nombre" required>
            <label>Numero de habitacion</label>
            <input type="text" name="habitacion" placeholder="Ej. 204" required>
            <div class="check-row">
                <input type="checkbox" id="t" required>

```

```

        <span>Acepto los <a style="color:#c9a96e88">terminos de uso</a> de
la red del hotel.</span>
    </div>
    <button class="btn" type="submit">Conectar</button>
</form>
    <p class="pie">Velocidad hasta 20 Mbps &bull; Soporte: ext. 0</p>
</div>
</body>
</html>
)rawliteral";

```

```

// Pagina de bienvenida que se muestra despues de llenar el formulario
// Los marcadores %NOMBRE% y %HABITACION% se reemplazan con los datos del
usuario antes de enviarse

```

```

const char BIENVENIDA_HTML[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0,
maximum-scale=1.0">
    <title>Conectado</title>
    <style>
        *{margin:0;padding:0;box-sizing:border-box}
        body{
            background:#09090f;min-height:100vh;
            display:flex;align-items:center;justify-content:center;
            font-family:-apple-system,'Helvetica Neue',sans-serif;
            color:#e8e0d0;padding:24px;
        }
        .card{
            background:#12121a;border:1px solid #2a2620;
            border-radius:20px;padding:44px 28px;
            width:100%;max-width:360px;text-align:center;
        }
        .ok{
            width:64px;height:64px;margin:0 auto 24px;
            background:#c9a96e18;border:1px solid #c9a96e55;
            border-radius:50%;display:flex;align-items:center;justify-
content:center;
        }
        .ok svg{width:30px;height:30px;fill:none;stroke:#c9a96e;stroke-
width:2.5;stroke-linecap:round;stroke-linejoin:round;}

```



```

        h1{font-family:Georgia,serif;font-size:20px;font-
weight:normal;color:#c9a96e;letter-spacing:.1em;margin-bottom:10px;}
        p{font-size:13px;color:#5a5040;line-height:1.6;}
        .nombre{color:#c9a96e;font-size:17px;font-weight:600;margin:14px 0 4px;}
        .hab{font-size:11px;letter-spacing:.15em;text-
transform:uppercase;color:#3a3020;}
    </style>
</head>
<body>
    <div class="card">
        <div class="ok">
            <svg viewBox="0 0 24 24"><polyline points="20 6 9 17 4 12"/></svg>
        </div>
        <h1>Bienvenido</h1>
        <p class="nombre">%NOMBRE%</p>
        <p class="hab">Habitacion %HABITACION%</p>
        <br>
        <p>Ya tienes acceso a Internet.<br>Disfruta tu estadia.</p>
    </div>
</body>
</html>
)rawliteral";

// Responde con el portal cuando alguien hace cualquier peticion a la raiz
del servidor
void manejarRaiz() {
    server.send(200, "text/html", PORTAL_HTML);
}

// Esta funcion se ejecuta cuando el usuario envia el formulario del portal
// Toma el nombre y el numero de habitacion, los mete en el HTML de
bienvenida y lo manda
void manejarConectar() {
    String nombre      = server.arg("nombre");
    String habitacion = server.arg("habitacion");

    // Se copia el HTML base y se reemplazan los marcadores con los datos del
usuario
    // Si el campo llegara vacio, se pone un valor por defecto para que no
quede en blanco
    String pagina = BIENVENIDA_HTML;
    pagina.replace("%NOMBRE%",      nombre.length()    > 0 ? nombre      :
"Huesped");

```

```

    pagina.replace("%HABITACION%", habitacion.length() > 0 ? habitacion : "---");

    // Se imprime en el monitor serial para poder ver quien se conecto durante
    las pruebas
    Serial.println("Nuevo acceso al portal:");
    Serial.println("    Nombre:    " + nombre);
    Serial.println("    Habitacion: " + habitacion);

    server.send(200, "text/html", pagina);
}

// Si alguien intenta cargar una URL que no esta registrada, se le redirige
al portal
// Esto es lo que hace que el portal aparezca solo en el dispositivo al
conectarse
void manejarNoEncontrado() {
    server.sendHeader("Location", "http://192.168.4.1/", true);
    server.send(302, "text/plain", "");
}

void setup() {
    Serial.begin(115200);
    delay(500);

    Serial.println("Iniciando portal cautivo...");

    // Paso 1: Se configura el ESP32 en modo punto de acceso WiFi
    // softAPConfig define la IP del ESP32, el gateway y la mascara de subred
    WiFi.mode(WIFI_AP);
    WiFi.softAPConfig(IP_AP, IP_AP, IPAddress(255, 255, 255, 0));
    WiFi.softAP(SSID_RED, PASS_RED);

    Serial.println("Red WiFi activa: " + String(SSID_RED));
    Serial.println("IP del portal:    " + IP_AP.toString());

    // Paso 2: Se inicia el servidor DNS que redirige todo el trafico hacia el
    ESP32
    // El asterisco como dominio significa que va a responder a cualquier
    nombre que se consulte
    // Esto es lo que provoca que el celular muestre el portal automaticamente
    al conectarse
    dnsServer.start(DNS_PORT, "*", IP_AP);

```

```

    // Paso 3: Se registran las rutas del servidor web con su funcion
correspondiente
    server.on("/",          manejarRaiz);    // Ruta principal, muestra el
portal
    server.on("/conectar", manejarConectar); // Ruta del formulario, procesa
los datos
    server.onNotFound(manejarNoEncontrado); // Ruta comodin, redirige
cualquier otra URL

    server.begin();
    Serial.println("Servidor web listo en el puerto 80");
}

// El loop se ejecuta en ciclo continuo para mantener el sistema activo
void loop() {
    dnsServer.processNextRequest(); // Atiende las solicitudes del servidor
DNS
    server.handleClient();          // Atiende las solicitudes del servidor
web HTTP
}

```

9. Conclusiones

Eduardo Cadengo Lopez

Esta práctica me hizo entender de una forma muy directa cómo funciona el mecanismo de los portales cautivos que vemos en todos lados. Lo que más me llamó la atención es que no necesitas hardware costoso ni configuraciones complicadas para lograrlo. El ESP32 hace todo al mismo tiempo: crea la red, responde las peticiones DNS y sirve las páginas web, y todo eso corre en un microcontrolador pequeño y barato. Lo que aprendí es que la clave del portal cautivo no está en el servidor web sino en el DNS, porque es el que engaña al dispositivo para que vaya a la página del portal en lugar de a internet. Tener eso claro me ayuda a entender mucho mejor cómo funcionan las redes reales.

Damian Alexander Diaz Pina

Lo que más me gustó de esta práctica fue ver el resultado en tiempo real en el celular. Cuando te conectas a la red y el portal aparece solo, sin que hagas nada, se siente que algo "mágico" pasó, aunque ahora ya se exactamente por qué ocurre. Durante las pruebas note que el portal tardaba diferente dependiendo del sistema operativo: en Android aparece casi de inmediato, pero en iOS tardaba un poco más. Eso me hizo pensar que cada sistema tiene su propia forma de detectar si hay un portal cautivo en la red. En general la práctica me dejó claro que es posible

construir sistemas de red funcionales usando solo el ESP32, sin depender de Routers externos ni de internet.

Itzel Citlalli Martell De La Cruz

Antes de esta práctica no entendía bien porque el portal de los hoteles o aeropuertos aparece solo al conectarme, y ahora que lo hice desde cero tiene mucho más sentido. La parte que más me ayudo a entenderlo fue configurar el servidor DNS, porque ahí es donde está el verdadero truco: el dispositivo cree que está haciendo una consulta normal de internet, pero el DNS lo redirige al ESP32 sin que el usuario se dé cuenta. Me parece que este tipo de proyectos son muy útiles porque conectan lo que aprendes en teoría con algo que puedes ver funcionar de verdad. además, tiene aplicaciones reales inmediatas, no solo en hoteles sino en cualquier lugar donde se quiera controlar o personalizar el acceso a una red WiFi.

10. Enlaces del Proyecto y Evidencias

GitHub:

https://github.com/AresGodKiller/Tecnologias_Inalambricas/tree/main/ESP32_PortalCautivo

YouTube: <https://youtube.com/shorts/VcKZFpCU6pw?feature=share>



